Programmation procédurale (UAA11)

Atelier : fonctions et tableaux

Objectifs de l'atelier :

- ✔ Observer qu'une fonction peut modifier un tableau, mais pas un entier.
- ✔ Découvrir ce que reçoit réellement une fonction à laquelle on passe un tableau.
- ✔ Comprendre pourquoi il faut transmettre la taille d'un tableau.
- ✔ Remplir un tableau dans une fonction.
- ✔ Comprendre pourquoi une chaîne n'a pas besoin de sa taille.

Avant de commencer

Cet atelier se fait seul, avant la séance de théorie. Tu n'as rien à apprendre par cœur : tu vas compiler de petits programmes, les exécuter, les modifier, et découvrir par toi-même comment le C se comporte. Les règles que tu formuleras serviront de point de départ à la séance.

Les programmes à télécharger :

https://lucasembrechts.github.io/UAA11/fichiers/atelier_fonctions_tableaux.zip

Comment travailler :

- ➢ décompresse l'archive : tu obtiens un fichier `.c` par programme. Le nom du fichier est indiqué au-dessus de chaque programme ;
- ➢ ouvre chaque fichier dans ton éditeur habituel, compile-le et exécute-le ;
- ➢ **avant** chaque exécution, écris ta prédiction. Ne la corrige pas après coup : une prédiction fausse est justement ce qui va t'apprendre quelque chose ;
- ➢ certains programmes ne compilent pas : c'est voulu. Lis alors attentivement le message du compilateur, il fait partie de la découverte ;
- ➢ ne cherche pas la réponse dans le cours ou sur Internet : l'atelier est conçu pour que tu la trouves en observant ;
- ➢ à la fin de chaque section, résume ta découverte dans le cadre « Ma règle ». L'ensemble de tes règles formera ta fiche pour la séance théorique.

Section 0 : une surprise

Fichier : section0.c

```c
#include <stdio.h>

void remiseAZero(int n);
void remiseAZeroTableau(int t[]);

void main(void) {
    int score = 42;
    int scores[3] = {42, 17, 8};
    remiseAZero(score);
    remiseAZeroTableau(scores);
    printf("score = %d\n", score);
    printf("scores[0] = %d\n", scores[0]);
}

void remiseAZero(int n) {
    n = 0;
}

void remiseAZeroTableau(int t[]) {
    t[0] = 0;
}
```

Prédis, puis exécute

Qu'affiche ce programme ?

Explique

Les deux fonctions font la même chose : mettre une valeur à 0. Pourquoi une seule a-t-elle un effet dans `main` ? Propose une explication.

✎ Ma règle : qu'as-tu observé ?

Section 1 : ce que reçoit la fonction

Le code `%p` affiche une adresse en mémoire.

Fichier : section1.c

```
#include <stdio.h>

void montrer(int n, int t[]);

void main(void) {
    int score = 42;
    int scores[3] = {42, 17, 8};
    printf("main    : score  est a l'adresse %p\n", &score);
    printf("main    : scores est a l'adresse %p\n", scores);
    montrer(score, scores);
}

void montrer(int n, int t[]) {
    printf("montrer : n      est a l'adresse %p\n", &n);
    printf("montrer : t pointe vers l'adresse %p\n", t);
}
```

Prédis

Les adresses exactes changent d'un ordinateur à l'autre. Prédis seulement : parmi les quatre adresses affichées, lesquelles seront identiques ?

Exécute, puis explique

Que reçoit la fonction quand on lui passe `score` ? Et quand on lui passe `scores` ?

Explique maintenant la surprise de la section 0.

✎ Ma règle : comment une variable et un tableau sont-ils transmis à une fonction ?



Section 2 : la taille perdue

Fichier : section2.c

```
#include <stdio.h>

void mesurer(int t[]);

void main(void) {
    int scores[5] = {42, 17, 8, 23, 4};
    printf("main    : sizeof = %d\n", sizeof(scores));
    mesurer(scores);
}

void mesurer(int t[]) {
    printf("mesurer : sizeof = %d\n", sizeof(t));
}
```

Prédis

Qu'affiche ce programme ?

Compile, exécute, puis explique

Le compilateur t'a-t-il averti de quelque chose ?

Le tableau compte 5 cases de 4 octets. Pourquoi `mesurer` trouve-t-elle une autre valeur ? Que mesure-t-elle ?

Comment la fonction peut-elle alors savoir combien de cases elle doit parcourir ?

Modifie

Fichier : section2_taille.c

```
#include <stdio.h>
```

```
void afficher(int t[], int taille);

void main(void) {
    int scores[5] = {42, 17, 8, 23, 4};
    int bonus[2] = {7, 3};
    afficher(scores, 5);
    afficher(bonus, 2);
}

void afficher(int t[], int taille) {
    for (int i = 0; i < taille; i++) {
        printf("%d ", t[i]);
    }
    printf("\n");
}
```

Prédis l'affichage, puis exécute. Pourquoi peut-on utiliser la même fonction pour deux tableaux de tailles différentes ?

✎ Ma règle : pourquoi faut-il passer la taille d'un tableau à une fonction ?

Section 3 : remplir un tableau dans une fonction

Fichier : section3.c

```
#include <stdio.h>
#define NB 5

void remplirCarres(int t[], int taille);

void main(void) {
    int carres[NB];
    remplirCarres(carres, NB);
    for (int i = 0; i < NB; i++) {
        printf("%d ", carres[i]);
    }
    printf("\n");
}

void remplirCarres(int t[], int taille) {
    for (int i = 0; i < taille; i++) {
        t[i] = (i + 1) * (i + 1);
    }
}
```

Prédis, puis exécute

Qu'affiche ce programme ?

Explique

`remplirCarres` ne renvoie rien (`void`). Comment `main` obtient-il pourtant les carrés ?

Modifie

Fichier : section3_retour.c

```
#include <stdio.h>

int[] creerCarres(void);

void main(void) {
    int carres[5];
    carres = creerCarres();
}
```

Ce programme essaie qu'une fonction renvoie un tableau avec `return`. Compile-t-il ?

✎ Ma règle : comment une fonction peut-elle produire un tableau ?

Section 4 : et les chaînes ?

Fichier : section4.c

```
#include <stdio.h>

int compterE(char texte[]);

void main(void) {
    char phrase[] = "Le petit chene";
    printf("%d\n", compterE("elephant"));
    printf("%d\n", compterE(phrase));
}

int compterE(char texte[]) {
    int n = 0;
    for (int i = 0; texte[i] != '\0'; i++) {
        if (texte[i] == 'e') {
            n++;
        }
    }
    return n;
}
```

Prédis, puis exécute

Qu'affiche ce programme ?

Explique

`compterE` ne reçoit pas la taille de la chaîne. Comment sait-elle où s'arrêter ?

Pourrait-on faire la même chose avec un tableau d'entiers ?

Modifie

Fichier : section4_masquer.c

```
#include <stdio.h>

void masquerE(char texte[]);
```

```
void main(void) {
    char phrase[] = "Le petit chene";
    masquerE(phrase);
    printf("%s\n", phrase);
}

void masquerE(char texte[]) {
    for (int i = 0; texte[i] != '\0'; i++) {
        if (texte[i] == 'e') {
            texte[i] = '*';
        }
    }
}
```

Prédis l'affichage, puis exécute. La chaîne de `main` a-t-elle été modifiée ? Pourquoi ?

✎ Ma règle : qu'est-ce qui change quand on passe une chaîne à une fonction ?

Défi final

Le fichier `defi.c` contient déjà `main` et les prototypes. Écris les cinq fonctions, sans modifier `main`.

Fichier : defi.c

```
#include <stdio.h>
#define NB_ELEVES 6

void saisirNotes(int notes[], int taille);
float moyenne(int notes[], int taille);
int compterReussites(int notes[], int taille, int seuil);
void ajouterBonus(int notes[], int taille, int bonus);
void afficherNotes(int notes[], int taille);

void main(void) {
    int notes[NB_ELEVES];
    saisirNotes(notes, NB_ELEVES);
    afficherNotes(notes, NB_ELEVES);
    printf("Moyenne : %.2f - Reussites : %d\n", moyenne(notes, NB_ELEVES),
compterReussites(notes, NB_ELEVES, 10));
    ajouterBonus(notes, NB_ELEVES, 2);
    afficherNotes(notes, NB_ELEVES);
    printf("Moyenne : %.2f - Reussites : %d\n", moyenne(notes, NB_ELEVES),
compterReussites(notes, NB_ELEVES, 10));
}

// À toi d'écrire les cinq fonctions ici.
```

- `saisirNotes` demande les notes à l'utilisateur ;
- `moyenne` renvoie la moyenne des notes ;
- `compterReussites` renvoie le nombre de notes supérieures ou égales au seuil ;
- `ajouterBonus` ajoute un bonus à chaque note, sans jamais dépasser 20 ;
- `afficherNotes` affiche les notes sur une ligne.

Exemple d'exécution :

```
Note 1 : 12
Note 2 : 8
Note 3 : 19
Note 4 : 9
Note 5 : 15
Note 6 : 6
Notes : 12 8 19 9 15 6
Moyenne : 11.50 - Reussites : 3
Notes : 14 10 20 11 17 8
Moyenne : 13.33 - Reussites : 5
```

Lesquelles de tes fonctions modifient le tableau ? Lesquelles se contentent de le lire ?

Pourquoi `moyenne` peut-elle renvoyer son résultat avec `return`, alors que `saisirNotes` ne le peut pas ?